SC4052 Cloud Project Report

Lee Jun Yang
Student ID: U2220336C

April, 5 2026

1 Introduction

The use of large language models (LLMs) enables systems to perform tasks when provided with sufficiently detailed prompts. However, despite their capabilities, LLMs remain general-purpose models rather than domain-specific specialists. Models such as ChatGPT and Gemini are typically equipped with general tools that allow them to assist with a wide range of tasks, but they are not inherently designed to execute complex, multi-step workflows in a structured and reliable manner.

To address this limitation, agentic system architectures have been developed. These architectures enhance LLMs by integrating external tools, memory, and orchestration mechanisms, enabling them to perform tasks in a more structured and goal-directed manner. Instead of relying on a single model, agent-based systems decompose objectives into smaller subtasks, delegate them to specialized services, and coordinate execution through well-defined interfaces.

For instance, agentic systems such as **OpenClaw** and **Claude Code** utilise agent-based architectures to deliver more effective and context-aware results for users.

This project explores the design of a PA-as-a-Service (PAaaS) platform based on cloud microservices, focusing on decomposing assistant capabilities into independent services and demonstrating how AI agents can coordinate these services through RESTful API interactions.

2 Literature Review

This section reviews existing agentic system architectures to provide context for the design of PAaaS platforms.

2.1 Orchestrator-Worker Architecture (OpenClaw)

OpenClaw employs an orchestrator-worker architecture in which a central orchestrator delegates tasks to specialised worker modules. Each worker handles a specific subtask, such as scheduling, email management, or preference tracking. This modular approach enables efficient coordination of multiple services and ensures scalability and maintainability in a cloud environment.

In addition, OpenClaw adopts a composable prompt architecture, where multiple configuration files and dynamic context sources are combined to construct the final system prompt sent to the model. Contextual elements such as tool definitions, agent instructions, personality configuration, and chat history are incorporated into this prompt.

All of these components are integrated with the base system instructions from the agent runtime (Pi Agent Core) to form the final system prompt. This composable design allows developers to modify agent behaviour, tone, and task capabilities through configuration files without changing the underlying source code, while still maintaining execution constraints and security boundaries.

Externally, OpenClaw exposes interfaces such as user interfaces, communication channels, and application programming interfaces (APIs) over network protocols (e.g., HTTP and WebSocket). Internally, however, agent logic and tool execution are handled through direct in-process function calls rather than through network-based communication.

2.2 Claude Code Architecture (Leaked)

Following the recent leak of Anthropic's Claude Code source code, it is possible to analyse its agentic architecture in greater detail. Claude Code utilises an agent-based design in which multiple specialised components interact under a central orchestrator to execute complex workflows. This architecture demonstrates effective task decomposition and service orchestration, providing insights into real-world implementations of agentic systems.

Referring to https://ccunpacked.dev/ and an analysed snapshot of a publicly exposed source code repository, Claude Code implements a coordinator-worker architecture in which a central coordinator agent orchestrates multiple worker agents to perform complex tasks. The coordinator is responsible for task decomposition, delegation, and result synthesis, while worker agents execute subtasks using a restricted set of tools.

Each worker is equipped with access to specific tools such as file operations, command execution, and search utilities. Tool access is managed through a permission layer and can be dynamically configured based on environment settings. This ensures controlled execution and reduces the risk of unintended actions.

Communication between the coordinator and workers follows a message-based pattern, where workers return structured results upon task completion. The coordinator interprets these results and decides whether to spawn additional workers, continue existing ones, or finalise the task.

Additionally, the system aggregates relevant contextual information, including system-level data (e.g., Git status and feature flags) and user-specific memory (e.g., project notes and cached context), prior to inference. This context is injected into the prompt, ensuring that the model operates with up-to-date environmental and user-specific information when generating responses.

3 Methodology and System Design

3.1 Technology Stack Used

3.1.1 Model Used

The system utilises Qwen 3 (latest version).

3.1.2 Backend and Orchestration

- **Python**: Chosen as the primary programming language due to its extensive ecosystem for artificial intelligence and LLM integration.
- **FastAPI**: Utilised to develop a high-performance, asynchronous RESTful API. Its native support for concurrency is critical for enabling efficient interactions between the user interface and backend services.
- **LangGraph**: Serves as the core orchestration engine. It manages directed graph execution and state transitions, enabling conditional routing and cyclic execution. These capabilities support the implementation of human-in-the-loop (HITL) feedback loops and iterative refinement of artefacts.
- **LangChain**: Provides the underlying framework for agentic chains, memory management, and prompt templating, ensuring that agents can maintain context across interactions.
- **Ollama**: A local runtime used to host and run LLMs on-device.

3.1.3 Data Persistence and Storage

- **SQLite / PostgreSQL**: Used as relational databases to store structured artefacts and inter-agent communications within the agentic pipeline. They also provide state persistence for workflow continuation and history tracking. SQLite is adopted for prototyping purposes, whereas PostgreSQL is used in production environments.

3.1.4 Frontend

- **React (Vite)**: Used to develop a responsive and interactive user interface. This enables users to interact effectively with the pipeline and visualise generated artefacts.

3.2 Architecture Used

To implement this PAaaS application, and considering that this is a small-scale prototype, predefined agents are used rather than relying on a general worker with dynamically injected tools during the agentic pipeline.

A hybrid architecture is adopted based on the models discussed in Section 2, combining an orchestrator-worker pipeline with a service-oriented design. To balance the high-speed requirements of an intelligent orchestrator with the decoupled nature of a PAaaS platform, components are initially implemented as scalable services organised as separate routers within a single FastAPI application. This structural choice establishes strict RESTful API contracts for each capability, ensuring the architecture is inherently cloud-ready and logically independent. Furthermore, to explicitly demonstrate the orchestration of independent services via well-defined RESTful API calls, core state-altering workflows are physically decoupled, for instance, both the email service and the user preference service are exposed and invoked via independent HTTP endpoints (e.g., POST /services/email/invoke). Ultimately, this dual-path design reduces operational overhead for prototyping while successfully proving that the central agent can coordinate distributed RESTful APIs to execute tasks and maintain an evolving model of user preferences

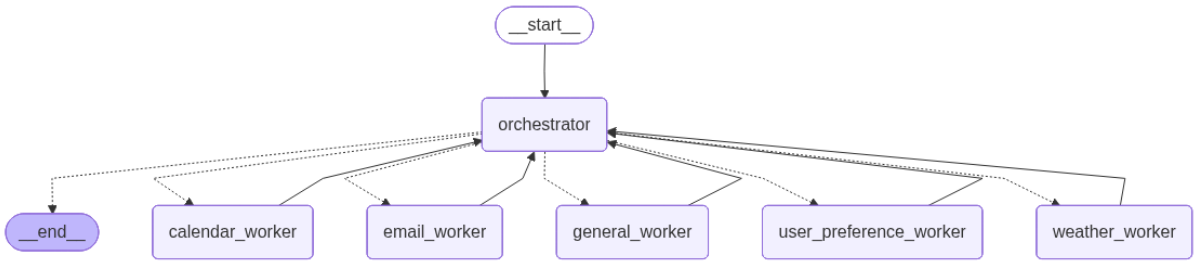

Figure 1: Overall Architecture

With reference to Figure 1, the system consists of five specialised agents, each responsible for a specific function. User input is first processed and passed to the orchestrator node. At this stage, the system constructs the system prompt by aggregating relevant configuration components such as `agents.md`, `rules.md`, and `preference.md` (if available, based on user-provided preferences).

Once the prompt is assembled, it is sent to the LLM along with the user input. The LLM, acting as the orchestrator, determines which agent should handle the request and routes the task accordingly.

3.3 Hybrid End Point Approach

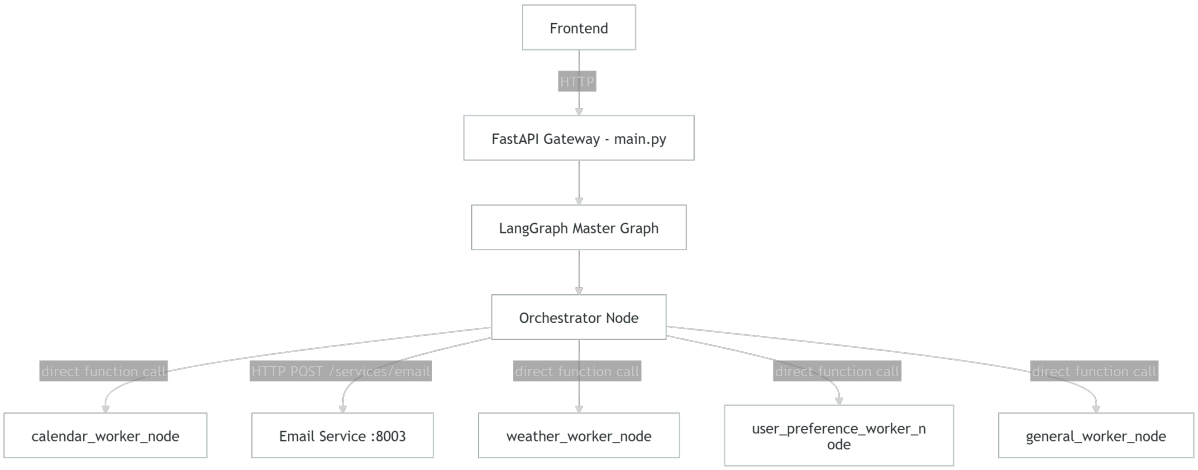

Figure 2: Service-Endpoint Architecture

With refernce to Fielding, 2000 our system exposes a set of RESTful endpoints that support both health monitoring and conversational interaction. The `GET /health` endpoint is used to verify that core infrastructure components—such as the graph, database connection, and checkpointer—are correctly initialised. It returns a `200 OK` status if all components are healthy, and a `503 Service Unavailable` status if any component is missing or fails.

The primary interaction begins with `POST /api/thread?user_id=xxx`, which initialises a new conversation session by generating a unique `thread_id`. This step corresponds to the first stage of the chat flow and returns an initial onboarding message prompting the user to configure their preferences.

The main chat functionality is handled by `POST /api/chat/stream`, which serves as the core endpoint of the system. It accepts a request containing `thread_id`, `user_id`, and the user message, and returns a Server-Sent Events (SSE) stream. Upon receiving a request, the system processes the input through the agentic graph, where the orchestrator routes the task to the appropriate worker agent, which then executes the required operations. The results are streamed back incrementally to the frontend.

The SSE stream includes multiple event types to reflect the system's internal state and progress. These include `node`, indicating the currently active graph node (e.g., orchestrator or a specific worker); `route`, representing the orchestrator's routing decision; `message`, containing the generated response content; `interrupt`, indicating a human-in-the-loop (HITL) approval requirement for sensitive actions such as calendar modifications or email operations; `error`, signalling execution failures; and `done`, marking the completion of the response stream.

The hybrid approach, as illustrated in Figure 2, exposes only the email service via an HTTP endpoint, while the remaining agents are invoked and routed through direct function calls within the orchestrator graph. To support decoupling and scalability of individual components, the email service is made accessible through a service-based HTTP interface via the `POST /services/email/invoke` endpoint. This design ensures that the email service can be scaled or fail independently without affecting the overall system, thereby improving fault isolation and system robustness.

This dual-path design allows the same agent logic to operate both within the orchestrated graph and as an independent service, while maintaining consistent behaviour across different execution contexts. In this design, only the email agent is exposed as an independent, network-accessible service, whereas the other agents remain tightly integrated within the orchestrator and are not exposed as separate endpoints.

4 Platform Functionalities

Our application is a small-scale PAaaS platform that enables users to read, compose, and manage emails, handle calendar events, and access real-time weather information through a conversational interface.

At its core, the system functions as an intelligent digital assistant capable of managing communications and scheduling tasks while integrating real-time external data sources. The platform supports email operations such as reading, searching, drafting, and replying, as well as calendar management with capabilities including event creation, retrieval, updates, and deletion. In addition, it provides access to detailed weather data, including temperature, humidity, wind speed, rainfall, UV index, and air quality indicators such as the PSI, along with multi-day forecasts.

4.1 Core Capabilities

1. Intelligent Email Management

- **Contextual Reading & Searching:** The system can query and summarise emails or locate specific messages within the inbox.
- **Drafting & Responding:** Users can instruct the assistant to compose new emails or reply to existing threads, incorporating user-defined preferences such as tone and formatting.

2. Proactive Calendar Scheduling

- **Event Lifecycle Management:** Supports full CRUD (create, read, update, delete) operations for calendar events.
- **Conflict Resolution:** Automatically checks for scheduling conflicts and validates availability before creating events.

3. Real-Time Meteorological Intelligence

- **Local & Global Forecasting:** Provides up-to-date weather conditions across regions.
- **Environmental Insights:** Includes metrics such as UV index, PSI, and extended multi-day forecasts.

4. Persistent User Personalisation

- **Tailored Experience:** A dedicated preference agent maintains user-specific settings such as timezone, meeting buffers, preferred email tone, and temperature units, ensuring consistent behaviour across sessions.

5 Agent Functionalities

Referring to Table 1, the tools available to each agent during the agentic pipeline are listed.

Agent	**Worker Name**	**Available Tools**
Calendar Agent	calendar_worker	get_calendar_events create_calendar_event delete_calendar_event conflict_calendar_event find_events_by_name delete_selected_event
Email Agent	email_worker	read_emails search_emails send_email reply_to_email
Weather Agent	weather_worker	get_two_hour_forecast get_temperature get_humidity get_wind_speed get_rainfall get_uv_index get_psi get_four_day_outlook get_worldwide_weather
User Preference Agent	user_preference_worker	create_md_preference
General Agent	general_worker	None (natural language responses only)

Table 1: Agents and their corresponding workers with available tools

6 Implementation

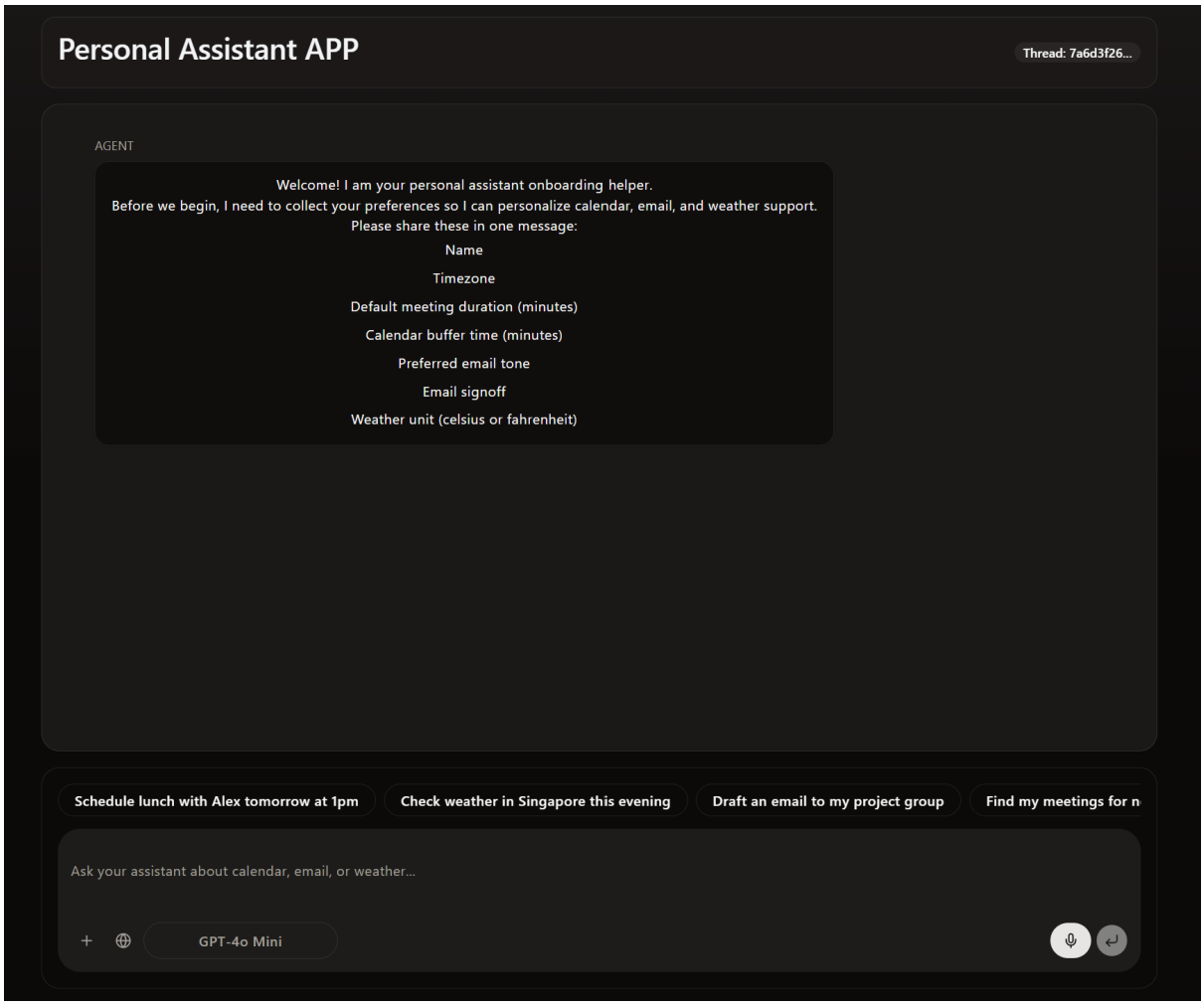

Figure 3: User Interface (UI)

Figure 3 shows a snapshot of how the user interface appears. The application first prompts the user to input personalised details so that the output can be tailored accordingly.

Figure 4: Weather Questions in Tokyo

Figure 4 shows an example of a weather query interaction within the application.

6.1 Orchestrator and Worker Workflow

Figure 5 illustrates the interaction between the orchestrator and worker agents within the system. The user input is first processed by the orchestrator, which is responsible for interpreting

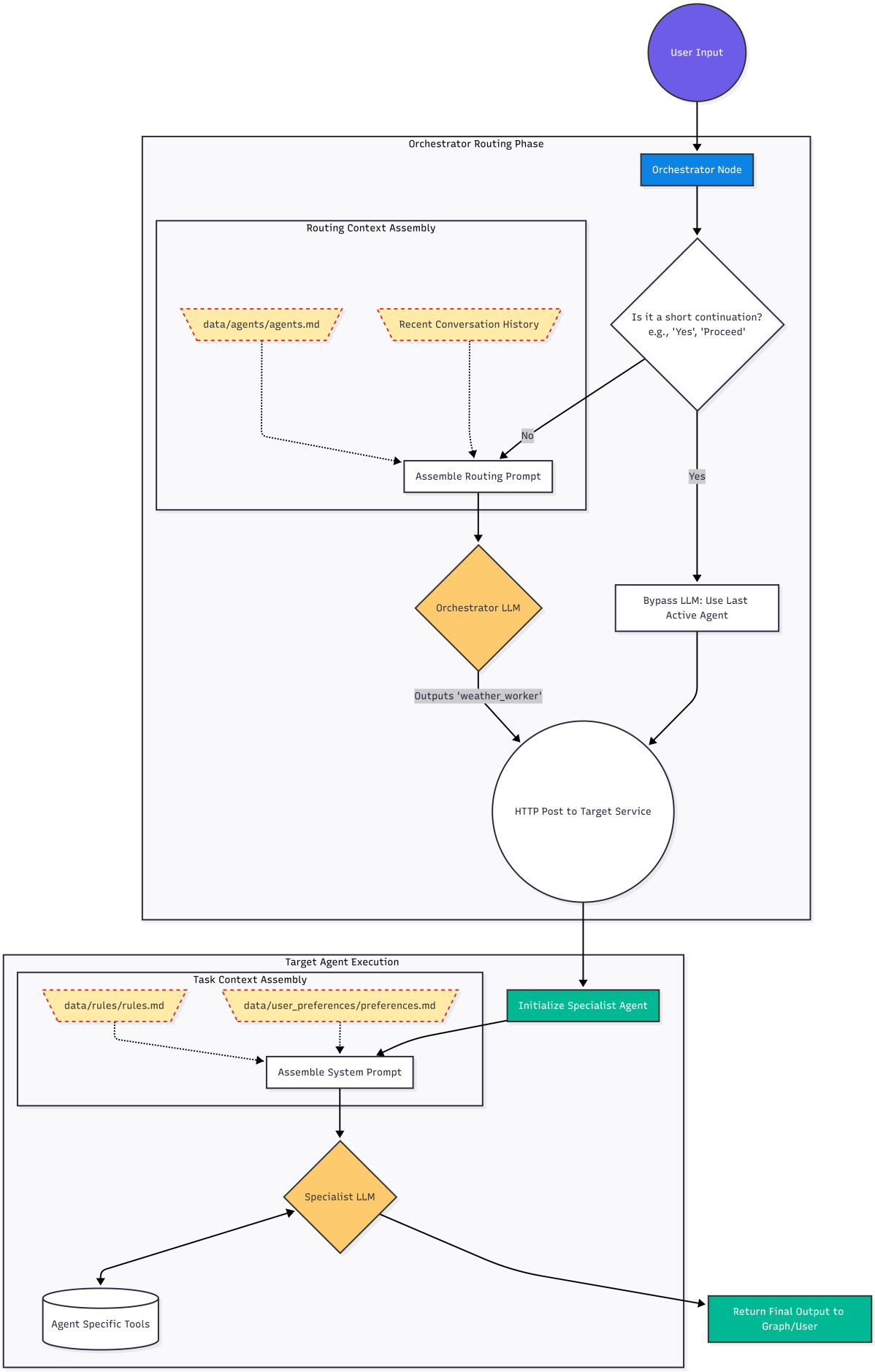

Figure 5: Overall Workflow

the request and determining the appropriate worker to handle the task. This process aligns with the approach described in the literature review, where system prompts are constructed prior to being passed to the LLM for response generation.

Before the orchestrator LLM is invoked, contextual information such as `agents.md` and the recent conversation history are assembled to form the routing prompt. This enables the orchestrator to make informed routing decisions based on both predefined agent capabilities and prior interactions.

Once a specialised agent is selected, its execution context is constructed by combining `rules.md`, `preferences.md`, and the relevant agent-specific tools. These components are integrated into the system prompt for the specialist LLM, which then performs the required task and generates the final output.

6.2 Workflow Diagram Weather Query Example

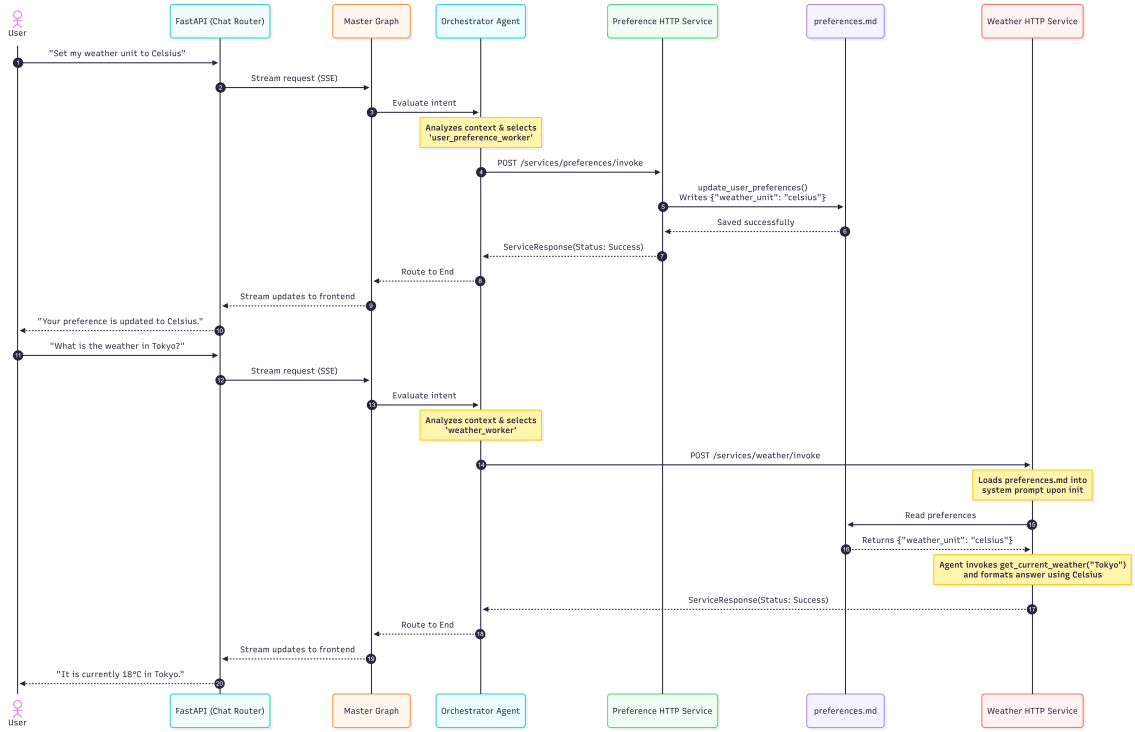

Figure 6: Workflow Diagram

The sequence diagram in Figure 6 illustrates the interaction between the user, the chat API, the master graph orchestrator, and external services.

In the first flow, the user updates preferences (e.g., setting the weather unit). The request is processed by the orchestrator, which routes it to the user preference worker. This worker invokes the Preference Service via an HTTP endpoint to update and persist the user settings. A confirmation message is then returned to the user through the streaming interface.

In the second flow, the user requests weather information. The orchestrator routes the request to the weather worker, which calls the Weather Service via an HTTP endpoint. The service loads the stored user preferences and generates a response consistent with the configured settings (e.g., temperature in Celsius), which is then streamed back to the user.

Overall, the diagram demonstrates the hybrid execution model, where the orchestrator handles routing while services execute the tasks.

6.3 Limitations

Due to resource constraints, smaller-sized LLMs were used in this application as the models were required to run locally on available GPUs. As a result, the token throughput is relatively

lower, which impacts the overall responsiveness and efficiency of the system.

7 Conclusion

In this project, a PAaaS platform was designed and implemented using a hybrid architecture that combines an orchestrator-worker model with service-oriented components. This approach enabled the decomposition of complex functionalities, such as email management, into independent services, while maintaining tighter integration for other agents through direct function calls within the orchestrator graph.

The system exposes a RESTful API layer that allows the frontend to interact with the backend, while internal agent workflows are managed through an orchestrated pipeline that handles routing, execution, and streaming of responses. This design provides a balance between modularity, scalability, and low communication overhead.

Overall, the implementation demonstrates how agentic systems can be structured to support multiple specialised agents, persistent user preferences, and real-time interaction within a unified conversational platform.

8 Appendix

8.1 Source Code

The source code for this project is available at the following repository:
https://github.com/JYL480/SC4052_Cloud_Project

References

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine]. https://www.ics.uci.edu/~fielding/pubs/dissertation/fielding_dissertation.pdf